Null coalesce works like `isset()` – `$val = $notDelared ?? 'default'` is logically equivalent to `$val = (isset($notDeclared)) ?: 'default';` – which means that if `$notDeclared` is, well, not declared then no Warning error is emitted. Null coalesce is also very useful for array handling because it will check that keys exist; if the key doesn't exist, then a default value can be substituted:

```
$arr = ['foo' => 'bar'];
print $arr['qux'] ?? 'something else';
```

```
Output:
something else
```

Null coalesce can (since PHP 7.4) also be combined with assignment like this – `??=`. This means that if the left-hand variable has a `null` value, then assignment takes place; otherwise, it doesn't:

```
$val1 = 'something';
$val2 = null;

$val1 ??= 'new val';
$val2 ??= 'new val';

var_dump($val1);
var_dump($val2);
```

```
Output:
string(9) "something"
string(7) "new val"
```

Null-Safe

When it comes to object access, there is an equivalent implicit logical test that has very similar syntax to the above. This is the *null safe* operator and is exclusively used with objects and their properties and methods. This allows for object calls, or property access, to safely fail and return `null` if the properties are null or the method returns null (the initial properties and methods need to exist, though):